LabGenius S Series Transfer Robot

Automation Integration Manual



- Version：V1.0
- Applicable Models：LabGenius S Series Transfer Robots

Bioyond Robotics(Shanghai)Co., Ltd

Contents

1. PREFACE 2

1.1. Copyright Statement 2
1.2. Scope of Application 2

2. SYSTEM STRUCTURE 3

2.1. System Diagram 3
2.2. System Varibales: Controlling Robot Motion 3
2.3. System Parameters: Setting Target Point Information 5

3. EXAMPLE SCENARIO 1: POINTS IN BIOYOND WORKS 6

3.1. Robot Program Explanation 7
3.2. Automation Control Software Program Explanation 11

4. EXAMPLE SCENARIO 2: POINTS IN THE AUTOMATION CONTROL PROGRAM 14

4.1. Robot Program Explanation 14
4.2. Automation Control Software Program Explanation 16

5. SWITCHING BETWEEN DIFFERENT APPLICATION SCENARIOS 21

1. Preface

Dear User,

Thank you for choosing the LabGenius S Series Transfer Robot manufactured by Bioyond Robotics(Shanghai) Co., Ltd.

This manual aims to help you quickly master the usage of the robot API interface, enabling efficient interaction with the robot. Please read the relevant materials carefully to make the most out of this product.

If you have any questions or needs during use, please feel free to contact us via phone or email, or visit our website. We are committed to providing you with the best service.

1.1. Copyright Statement

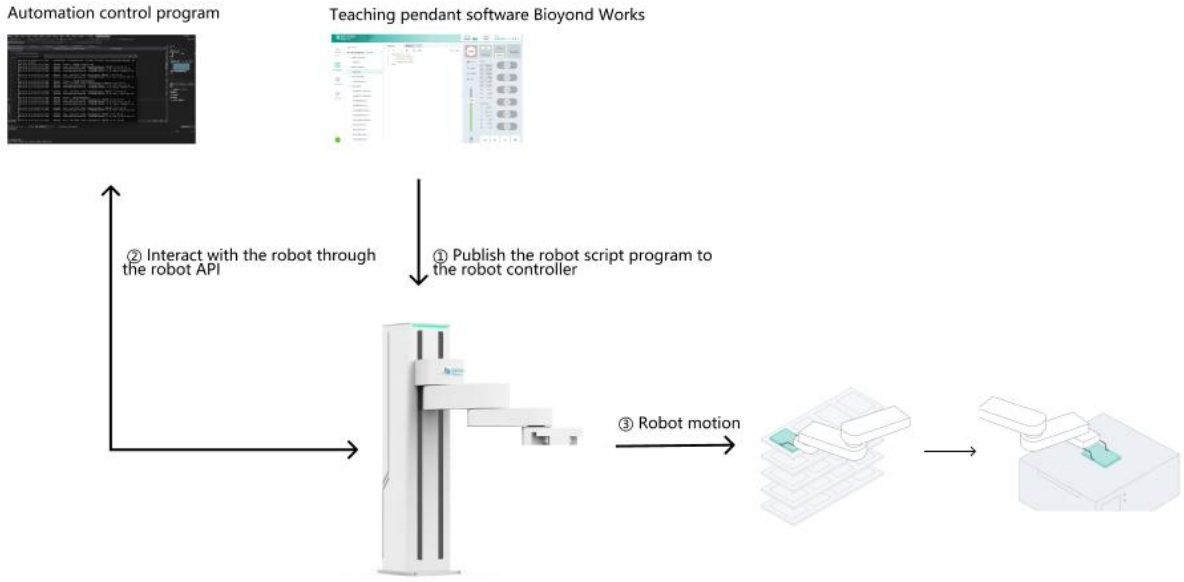
- All photos and illustrations mentioned in this publication are the property of Bioyond Robotics(Shanghai) Co., Ltd, and are protected by law.
- The information in this document is subject to change without notice. The manufacturer makes no representations or warranties, express or implied, regarding the content of this document for a particular purpose.
- Without prior written permission, this document may not be reproduced, copied, or translated into other languages.
- Contents herein may be revised without further notice.
- In case of discrepancies between illustrations and actual product appearances, the actual product prevails.
- This manual does not represent any specifications or hardware/software configurations of the product. For detailed specifications and configurations, please refer to the relevant product agreements, packing list, or consult with the sales department.
- Every effort has been made to ensure the accuracy and completeness of the contents in this manual. However, errors or omissions may still exist. Bioyond Robotics(Shanghai) Co., Ltd. continuously strives to optimize and improve its products and services. Therefore, we reserve the right to modify the product and its description in this manual at any time without prior notice. If you find any discrepancies between the actual product and the manual or need the latest information, please feel free to contact us or visit our website for inquiries.

1.2. Scope of Application

- This manual applies exclusively to LabGenius S Series Transfer Robots manufactured by Bioyond Robotics(Shanghai) Co., Lt

2. System Structure

2.1. System Diagram



- Bioyond Works

Before programming with the robot API interface, users need to use Bioyond Works software to write and deploy the Lua program. The Lua program primarily includes the robot motion control program and a TCP socket server program for client interaction. We provide Lua example programs, which users can modify based on their specific business requirements.

- Automation Control Program

After completing the writing and deployment of the Lua program, users can develop client programs via the robot API interface to achieve automated control of the robot. This API is based on TCP Socket communication with JSON data format and supports various programming languages, such as C/C++, C#, Python, and others. For detailed interface specifications, please refer to the document “Bioyond Robot Specification - Rob INTF_V1.0”. The automation control program can read and write system variables through the robot API interface to interact with the robot program. By setting system variable values, users can control the robot to execute the corresponding program, and by reading the variable values, they can determine the robot's motion status, thus enabling automated control.

2.2. System Varibales: Controlling Robot Motion

The automation control software sets system variables (such as SYSINVAR0, SYSINVAR1, etc.) by sending commands to the robot. The robot program performs conditional checks based on the values of these variables and executes the corresponding actions. This robot supports a maximum of 20 integer system variables (SYSINVAR0~SYSINVAR19). The automation control program and the robot program

need to agree on the usage rules of the system variables based on the application requirements, including:

1. The number of system variables used.
2. The meaning of each system variable.
3. The specific actions corresponding to different values.

For example, in the scenario mentioned in this document, the system variables are defined as follows:

System Variable	Meaning	Description
SYSINVAR0	TargetDeviceId	Represents different devices being operated.
SYSINVAR1	PosX	PosX, PosY, and PosZ represent the point indices on the device. For example, in stack-type devices, PosX can represent the row (1~m), PosY can represent the column (1~n), and PosZ can represent the layer (1~k), collectively describing m*n*k point locations.
SYSINVAR2	PosY	
SYSINVAR3	PosZ	
SYSINVAR4	RobotMoveType	Represents the robot's action type. For example: 1 for Pick, 2 for Place.
SYSINVAR5	RobotObjType	Represents the type of item the robot is handling. Different item types may affect parameters such as grip force and transfer speed. If not required, it can be ignored.
SYSINVAR6	PointInUpperComputer	Indicates whether the point is written by the upper computer. 0 means the point is written in the Bioyond Works, and 1 means the point is written in the upper computer. This needs to be used in conjunction with SYSINVAR7.
SYSINVAR7	MoveMode	When the point is written by the upper computer, it indicates the type of movement command. For example: 1 for MoveL, 2 for MoveJ, 3 for MoveAbsJ, 4 for MoveExtJ (open gripper), 5 for MoveExtJ (close gripper). This needs to be used in conjunction with SYSINVAR6.

Example 1: The automation control program sends a command to set the following system variable values, which means: The robot will place an item of material type '1' at device ID '1', to the XYZ position with coordinates (1, 1, 1). The point is written in the Bioyond Works

```
C#
SYSINVAR0 = 1
SYSINVAR1 = 1
SYSINVAR2 = 1
SYSINVAR3 = 1
SYSINVAR4 = 2
SYSINVAR5 = 1
SYSINVAR6 = 0
SYSINVAR7 = 0
```

Example 2: The automation control program sends a command to set the following system variable values, which means: The MoveAbsJ command is issued to the robot with parameters, moving the robot's joints to the target position. The point is written in the automation control program.

```
C#
SYSINVAR0 = 0
SYSINVAR1 = 0
SYSINVAR2 = 0
SYSINVAR3 = 0
SYSINVAR4 = 0
SYSINVAR5 = 0
SYSINVAR6 = 1
SYSINVAR7 = 3
```

The robot’s actual movement is determined by the robot program. We will explain this further with simple program examples.

2.3. System Parameters: Setting Target Point Information

Refer to the section 'Set floating point system parameter' in the document 'Bioyond Robot Specification - Rob INTF_v1.1' (hereinafter referred to as the Bioyond Robot Specification). The system parameters can be set by sending commands like the one below. System parameters can be integers or floating-point numbers. Example 2 uses floating-point numbers.

```
{
    "BRCAPI":"1.0",
    "cmd":"setSysParaF",
    "params":[
        {"index":0,"value":1.1},
        {"index":1,"value":2.2},
        {"index":2,"value":3.3}
    ],
    "id":123
}
```

Each index corresponds to a value, and the meaning of each value can be defined according to the actual application requirements. For example:
index0 represents the target position of Joint 1,
index1 represents the target position of Joint 2, and so on.
By setting the values corresponding to these indexes, the target points can be successfully written to the robot controller, enabling precise motion control.

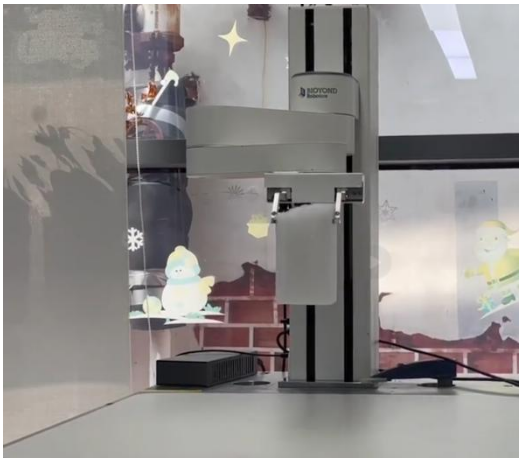
3. Example Scenario 1: Points in Bioyond Works

Scenario Description:

Using the robot to complete a sequence of picking up, moving, and placing objects. The robot starts from its initial position, moves to position 1 to pick up an object, places it at position 2, then picks up an object from position 2 and places it back at position 1, and finally returns to the initial position. The entire motion process is illustrated as follows:



(1)



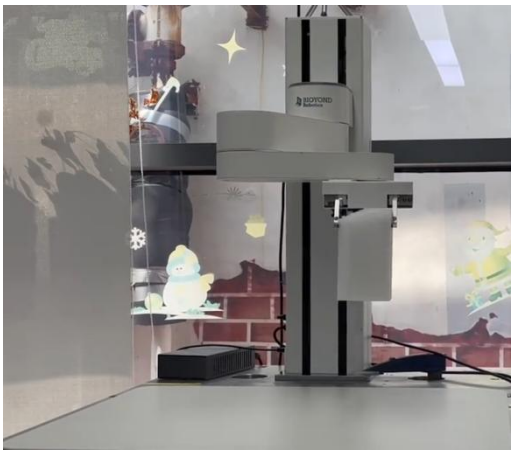
(2)



(3)



(4)



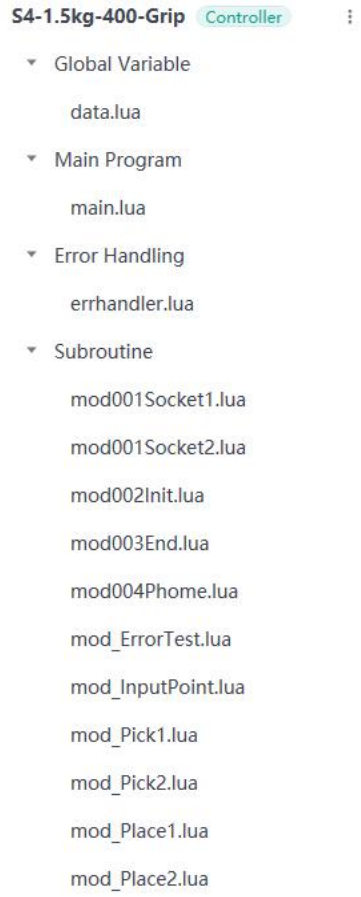


(5) (6)

The following is a detailed description of the example programs for the robot and the automation control software, tailored to the requirements of the scenario.

3.1. Robot Program Explanation

The robot program is written in Lua. The basic structure of the program file is as follows:



1. Main Program File: main.lua

main.lua is the entry point of the program, invoking the mod001Socket1 subroutine to enable communication with the automation control software and handle task scheduling.

```lua
function main()
  mod001Socket1()
end
```

2. Subprogram File

mod004Phome.lua: Return-to-Home Subprogram

Function: Defines and checks whether the robot is at the home point. If not, it moves the robot to the home point.

Note: This subprogram can be deleted if the home point is not required in the application.

```
Lua
function mod004Phome()
  while(1==1)do
    --LJ1 defines a user home point
    local LJ1=JointTarget(100,0,0,0,0,0,0,0,0,0,0,0)
    --LJ2 gets the current position
    local LJ2=GetCJoints()
    local LVAR1=0
    --Compare to check if the robot is at the home point
     LVAR1=Check4JointsEqual(LJ1,LJ2,5)
     if(LVAR1>=1)then
       ShowInfo("The Robot at home point")
       break
    --If not, move it to the home point through the MoveAbsJ
    else
       ShowInfo("The Robot is not at home point")
       MoveAbsJ(LJ1,50,0)
    end
  end
end
```

mod002Init.lua: Initialization Subprogram

Function: Initializes system variables and the Lua application state.

Note: Sets system input variables to their initial values and resets system output variables.

```
Lua
function mod002Init()
  SetLuaAppSta(0) --Initialize Lua program state, indicating the robot is not ready to run the program
  --TargetDeviceId
  SetSysInVarI(0,0) --Initialize system variables
  --Pos X
  SetSysInVarI(1,0)
  --Pos Y
  SetSysInVarI(2,0)
  --Pos Z
  SetSysInVarI(3,0)
  --RobotMoveType
  SetSysInVarI(4,0)
  --RobotObjType
  SetSysInVarI(5,0)
  --PointInUpperComputer
  SetSysInVarI(6,0)
```

```
    --MoveMode
    SetSysInVarI(7,0)
    ResetSysOutVarI() --Reset all system output variables
    SetLuaAppSta(1) --Set Lua program status to 1, indicating the robot is ready to run the program
    ShowInfo("Init Successful!")
end
```

mod_Pick1.lua: Pick Material Subprogram

Function: Enables the robot to perform the action of picking up materials from a specified location.

Note: Defines four target points corresponding to approach, gripper opening, descent, and gripping actions.

```
Lua
function mod_Pick1()
    --Define the four target points used in this subroutine, representing a pick instruction that includes four actions, such as:
    --Moving to the approach point, Opening the gripper, Moving to the target point, Closing the gripper, etc.
    --The number and values of the target points are defined based on the actual usage scenario
    local LJ1=JointTarget(278.501,-16.501,-49.635,71.279,0,0,0.014,0,0,0,0,0)
    local LJ2=JointTarget(278.501,-16.501,-49.636,71.278,0,0,28.000,0,0,0,0,0)
    local LJ3=JointTarget(108.213,-16.501,-49.636,71.278,0,0,28.000,0,0,0,0,0)
    local LJ4=JointTarget(108.213,-16.501,-49.636,71.278,0,0,-0.000,0,0,0,0,0)
    local LVAR1=50
    MoveAbsJ(LJ1,LVAR1,0)
    MoveExtJ(LJ2,LVAR1,0,T0,U0)
    MoveAbsJ(LJ3,LVAR1,0)
    MoveExtJ(LJ4,LVAR1,0,T0,U0)
    MoveAbsJ(LJ1,LVAR1,0)
    --The automation control software waits for the robot's motion completion signal, which is returned by mod003End()
    mod003End()
end
```

mod003End.lua: Command Completion Subprogram

Function: Resets system variables and sends a command completion signal.

Note: Notifies the automation control software that the command has been completed using SendSysOutVarI(1, 1).

```
Lua
function mod003End()
    --Wait for the motion to complete
    MOVFINISH()
    --Setting the system variables to their initial state is done to ensure that the robot's behavior is controlled entirely by the automation control software, rather than being controlled by LUA
    --TargetDeviceId
    SetSysInVarI(0,0)
```

```
    --Pos X
    SetSysInVarI(1,0)
    --Pos Y
    SetSysInVarI(2,0)
    --Pos Z
    SetSysInVarI(3,0)
    --RobotMoveType
    SetSysInVarI(4,0)
    --RobotObjType
    SetSysInVarI(5,0)
    --PointUpperComputer
    SetSysInVarI(6,0)
    --MoveMode
    SetSysInVarI(7,0)
    --Set the motion completion reply and send it to the automation control software, allowing the software to exit the waiting state and send subsequent commands. Otherwise, the automation control software will continue to wait until a timeout occurs
    SendSysOutVarI(1,1)
    ShowInfo("Sianal end")
end
```

mod001Socket1.lua: Socket Communication and Task Scheduling Subprogram

Function: Orchestrates other subprograms to enable communication with the automation control software and manage task scheduling.

Note: Calls different subprograms (e.g., mod_Pick1, mod_Place1, etc.) based on the values of system variables.

```
Lua
function mod001Socket()
    mod004Phome()
    mod002Init()
    while(1==1)do
        GetSysVarI()    --Get the system variables sent by the automation control software, and perform different pick and place movements based on the instructions indicated by the system variables
        if(SYSINVAR0~=0 and SYSINVAR1~=0 and SYSINVAR2~=0)then
            if(SYSINVAR3~=0 and SYSINVAR4~=0 and SYSINVAR5~=0)then
                if(SYSINVAR0==1) then
                    if(SYSINVAR4==1)then
                        mod_Pick1()
                    elseif(SYSINVAR4==2)then
                        mod_Place1()
                    end
                elseif(SYSINVAR0==2) then
                    if(SYSINVAR4==1)then
                        mod_Pick2()
```

```
                elseif(SYSINVAR4==2)then
                    mod_Place2()
                end
            end
        else
            ShowInfo("Signal Error")
            break
        end
    end
    DelayMS(10)
  end
end
```

3.2. Automation Control Software Program Explanation

Below is an example program based on C#, which performs the following functions:

1. Creates a socket and establishes a connection with the robot.
2. Sends different system variables to the robot, enabling it to perform the corresponding motions.
3. Waits for the robot to send a status callback indicating 'motion completed.'
4. Closes the connection.

```
C#
using BR.ECS.DeviceDriver.Robot.BYRobotSample.models;
using BR.ECS.DeviceDriver.Robot.BYRobotSample;
using System;
using System.Collections.Generic;
using System.Diagnostics.Contracts;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace BR.ECS.DeviceDriver.Tests
{
    public class RobotTestMain
    {
        static void Main()
        {
            //Initialize and set up the socket connection (IP address, port number, etc.)
            BYRobotSample robotDemo = new BYRobotSample();
            SocketCommunication socketCommunication = new SocketCommunication();
            socketCommunication.Init("192.168.3.200", 56788);
            robotDemo.Connect(socketCommunication);
```

```
            Thread.Sleep(5000);

            //Example corresponding to the Lua program mod001Socket1
            //When the robot Lua program receives these system variables, it will perform different movements based on their values
            //The API will enter a waiting state, and after the robot completes its motion, the Lua program will send related commands to confirm whether the system variables were set successfully.
            while (true)
            {
                //Set system variables: SYSINVAR0 = 1, SYSINVAR1 = 1, SYSINVAR2 = 1, SYSINVAR3 = 1, SYSINVAR4 = 1, SYSINVAR5 = 2
                Dictionary<string, int> setSysValParamsModels1 = new Dictionary<string, int>
                {
                        { "0", 1 },
                        { "1", 1 },
                        { "2", 1 },
                        { "3", 1 },
                        { "4", 1 },
                        { "5", 2 },
                };
                // Set the system variables with the corresponding data
                robotDemo.SetSysVarI(setSysValParamsModels1, 5 * 60 * 1000);

                //Set system variables: SYSINVAR0 = 1, SYSINVAR1 = 2, SYSINVAR2 = 2, SYSINVAR3 = 2, SYSINVAR4 = 2, SYSINVAR5 = 2
                Dictionary<string, int> setSysValParamsModels2 = new Dictionary<string, int>
                {
                        { "0", 1 },
                        { "1", 2 },
                        { "2", 2 },
                        { "3", 2 },
                        { "4", 2 },
                        { "5", 2 },
                };
                robotDemo.SetSysVarI(setSysValParamsModels2, 5 * 60 * 1000);

                //Set system variables: SYSINVAR0 = 2, SYSINVAR1 = 2, SYSINVAR2 = 2, SYSINVAR3 = 2, SYSINVAR4 = 1, SYSINVAR5 = 2
                Dictionary<string, int> setSysValParamsModels3 = new Dictionary<string, int>
                {
                        { "0", 2 },
                        { "1", 2 },
                        { "2", 2 },
                        { "3", 2 },
                        { "4", 1 },
```

```
                        { "5", 2 },
                    };
                    robotDemo.SetSysVarI(setSysValParamsModels3, 5 * 60 * 1000);

                    //Set system variables: SYSINVAR0 = 2, SYSINVAR1 = 1, SYSINVAR2 = 1,
SYSINVAR3 = 1, SYSINVAR4 = 2, SYSINVAR5 = 2
                    Dictionary<string, int> setSysValParamsModels4 = new Dictionary<string, int>
                    {
                        { "0", 2 },
                        { "1", 1 },
                        { "2", 1 },
                        { "3", 1 },
                        { "4", 2 },
                        { "5", 2 },
                    };
                    robotDemo.SetSysVarI(setSysValParamsModels4, 5 * 60 * 1000);
                }

                Thread.Sleep(20000);

                //Close the connection with the robot
                robotDemo.Disconnect();
                Console.ReadLine();
            }
        }
}
```

4. Example Scenario 2: Points in the Automation Control Program

Scenario Description:
The automation control program directly provides the point and motion type, allowing the robot to reach the specified position and perform the corresponding action.

4.1. Robot Program Explanation

1. Main Program File: main.lua

main.lua is the entry point of the program, invoking the mod001Socket1 subroutine to enable communication with the automation control software and handle task scheduling.

```lua
function main()
   mod001Socket2()
end
```

2. Subprogram File

mod_InputPoint.lua: Input Point Handling Subprogram
Function: Processes system parameters sent by the automation control software to set spatial or joint variables and execute the corresponding motion commands.
Note:
Uses LP1 as the spatial variable for MoveL and MoveJ commands.
Uses LJ1 as the joint variable for MoveAbsJ and MoveExtJ commands.
Selects and executes different motion commands based on the values of SYSINVAR6 and SYSINVAR7.

```lua
function mod_InputPoint()
   local LP1=RobTarget(0,0,0,0,0,0,0,0,0,0,0,0)   --Define spatial variable for MoveL, MoveJ
   local LJ1=JointTarget(0,0,0,0,0,0,0,0,0,0,0,0)   --Define joint variable for MoveAbsJ, MoveExtJ
   local LVAR1=0   --Initializes a set of variables
   local LVAR2=0
   local LVAR3=0
   local LVAR4=0
   local LVAR5=0
   local LVAR6=0
```

```
  local LVAR7=0
  LVAR1 = GetSysParamF(0)     --Get target position of joint 1 or X position
  LVAR2 = GetSysParamF(1)     --Get target position of joint 2 or Y position
  LVAR3 = GetSysParamF(2)     --Get target position of joint 3 or Z position
  LVAR4 = GetSysParamF(3)     --Get target position of joint 4 or Rz position
  LVAR5 = GetSysParamF(4)     --Maximum robot joint speed (mm/s) for MoveL, or speed ratio for
MoveJ/MoveAbsJ/MoveExtJ (%)
  LVAR6 = GetSysParamF(5)     --Fusion region for trajectory length in MoveL, or fusion joint angle for
MoveJ/MoveAbsJ/MoveExtJ
  LVAR7 = GetSysParamF(6)     --Control external axis 1 (e.g., gripper) open/close in MoveExtJ
  if(SYSINVAR6==1)then
    if(SYSINVAR7==1)then        --If SYSINVAR7 is 1, execute MoveL
      SetPose(LP1,X,LVAR1)
      SetPose(LP1,Y,LVAR2)
      SetPose(LP1,Z,LVAR3)
      SetPose(LP1,RZ,LVAR4)
      MoveL(LP1,LVAR5,LVAR6,T0,U0)
    elseif(SYSINVAR7==2)then    --If SYSINVAR7 is 2, execute MoveJ
      SetPose(LP1,X,LVAR1)
      SetPose(LP1,Y,LVAR2)
      SetPose(LP1,Z,LVAR3)
      SetPose(LP1,RZ,LVAR4)
      MoveJ(LP1,LVAR5,LVAR6,T0,U0)
    elseif(SYSINVAR7==3)then     --If SYSINVAR7 is 3, execute MoveAbsJ
      SetElem(LJ1,1,LVAR1)
      SetElem(LJ1,2,LVAR2)
      SetElem(LJ1,3,LVAR3)
      SetElem(LJ1,4,LVAR4)
      MoveAbsJ(LJ1,LVAR5,LVAR6)
    elseif(SYSINVAR7==4)then     --If SYSINVAR7 is 4, execute MoveExtJ (open gripper)
      DisableGripperDet(1)          --Disable gripper detection before opening
      SetElem(LJ1,1,LVAR1)
      SetElem(LJ1,2,LVAR2)
      SetElem(LJ1,3,LVAR3)
      SetElem(LJ1,4,LVAR4)
      SetElem(LJ1,7,LVAR7)
      MoveExtJ(LJ1,LVAR5,LVAR6,T0,U0)
    elseif(SYSINVAR7==5)then     --If SYSINVAR7 is 5, execute MoveExtJ (close gripper)
      SetElem(LJ1,1,LVAR1)
      SetElem(LJ1,2,LVAR2)
      SetElem(LJ1,3,LVAR3)
      SetElem(LJ1,4,LVAR4)
      SetElem(LJ1,7,LVAR7)
      MoveExtJ(LJ1,LVAR5,LVAR6,T0,U0)
      EnableGripperDet(1)          --Enable gripper detection after closing, if needed
```

```
    end
  end
  DelayMS(1000)
  ShowInfo(" Sianal end")
  mod003End()      --After the automation control software sends system variables, it waits for the robot's motion completion reply
end
```

mod001Socket2.lua: Socket Communication and Task Scheduling Subprogram
Function: Orchestrates other subprograms to enable communication with the automation control software and manage task scheduling.
Note: Calls the mod_InputPoint subprogram based on the values of system variables to execute the corresponding motion.

```
Lua
function mod001Socket2()
  mod004Phome()
  mod002Init()
  while(1==1)do
    GetSysVarI()   --Get system variables sent by the automation control software
    if(SYSINVAR6~=0 and SYSINVAR7~=0)then
      mod_InputPoint()
    end
    DelayMS(10)
  end
end
```

4.2. Automation Control Software Program Explanation

Below is an example program based on C#.

```
C#
using BR.ECS.DeviceDriver.Robot.BYRobotSample.models;
using BR.ECS.DeviceDriver.Robot.BYRobotSample;
using System;
using System.Collections.Generic;
using System.Diagnostics.Contracts;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace BR.ECS.DeviceDriver.Tests
```

```
{
    public class RobotTestMain
    {
        static void Main()
        {
            //Initialize and set up the socket connection (IP address, port number, etc.)
            BYRobotSample robotDemo = new BYRobotSample();
            SocketCommunication socketCommunication = new SocketCommunication();
            socketCommunication.Init("192.168.3.200", 56788);
            robotDemo.Connect(socketCommunication);
            Thread.Sleep(5000);

            //Example corresponding to the Lua program mod001Socket2
            //When the robot Lua program receives these system variables, it performs different movements based on their values
            //The API enters a waiting state, and once the robot completes the motion, Lua will send related commands to verify if the system variables were successfully set
            while (true)
            {

                // The following are system parameters:
                // index0: Target position of joint 1
                // index1: Target position of joint 2
                // index2: Target position of joint 3
                // index3: Target position of joint 4
                // index4: For MoveL, this is the robot's maximum joint speed, in mm/s; for MoveJ, MoveAbsJ, MoveExtJ, this is the speed ratio relative to the maximum joint speed, in %
                // index5: For MoveL, this is the fusion region trajectory length; for MoveJ, MoveAbsJ, MoveExtJ, this is the fusion region joint angle
                // index6: Controls the opening and closing of external axis 1 (e.g., gripper) during MoveExtJ command
                Dictionary<string, float> setSysParaF1 = new Dictionary<string, float>
                {
                        { "0", 271.83f },
                        { "1", 44.08f },
                        { "2", -120.67f },
                        { "3", 80.16f },
                        { "4", 50f },
                        { "5", 20f },
                        { "6", 0f }
                 };

                // The following are system variables:
                // SYSINVAR0 to SYSINVAR5 have been used in the first example scenario; set them to 0 if not used in this example
```

```
                // SYSINVAR6: Enables the point transfer function from the upper computer
                // SYSINVAR7: Controls the motion type; 1 for MoveL, 2 for MoveJ, 3 for MoveAbsJ, 4 for MoveExtJ (gripper open), 5 for MoveExtJ (gripper close)
                Dictionary<string, int> setSysVal1 = new Dictionary<string, int>
                {
                        { "0", 0 },
                        { "1", 0 },
                        { "2", 0 },
                        { "3", 0 },
                        { "4", 0 },
                        { "5", 0 },
                        { "6", 1 },
                        { "7", 3 },
                };
                // // MoveAbsJ: Perform joint movement to the first target point
                robotDemo.SetSysParaF(setSysParaF1, 5 * 60 * 1000);
                robotDemo.SetSysVarI(setSysVal1, 5 * 60 * 1000);

                Dictionary<string, float> setSysParaF2 = new Dictionary<string, float>
                {
                        { "0", 276.53f },
                        { "1", 337.21f },
                        { "2", 310f },
                        { "3", -0.98f },
                        { "4", 50f },
                        { "5", 20f },
                        { "6", 0f }
                 };
                Dictionary<string, int> setSysVal2 = new Dictionary<string, int>
                {
                        { "0", 0 },
                        { "1", 0 },
                        { "2", 0 },
                        { "3", 0 },
                        { "4", 0 },
                        { "5", 0 },
                        { "6", 1 },
                        { "7", 1 },
                };
                // MoveL: Perform linear motion to the second target point
                robotDemo.SetSysParaF(setSysParaF2, 5 * 60 * 1000);
                robotDemo.SetSysVarI(setSysVal2, 5 * 60 * 1000);

                Dictionary<string, float> setSysParaF3 = new Dictionary<string, float>
                {
```

```
            { "0", 426.93f },
            { "1", -6.87f },
            { "2", 167.80f },
            { "3", 16.91f },
            { "4", 50f },
            { "5", 20f },
            { "6", 0f }
 };
Dictionary<string, int> setSysVal3 = new Dictionary<string, int>
{
            { "0", 0 },
            { "1", 0 },
            { "2", 0 },
            { "3", 0 },
            { "4", 0 },
            { "5", 0 },
            { "6", 1 },
            { "7", 2 },
};
//MoveJ: Perform joint movement to the third target point
robotDemo.SetSysParaF(setSysParaF3, 5 * 60 * 1000);
robotDemo.SetSysVarI(setSysVal3, 5 * 60 * 1000);

Dictionary<string, float> setSysParaF4 = new Dictionary<string, float>
{
            { "0", 268.53f },
            { "1", -65.81f },
            { "2", -65.6f },
            { "3", 127.95f },
            { "4", 50f },
            { "5", 20f },
            { "6", 0f }
 };
Dictionary<string, int> setSysVal4 = new Dictionary<string, int>
{
            { "0", 0 },
            { "1", 0 },
            { "2", 0 },
            { "3", 0 },
            { "4", 0 },
            { "5", 0 },
            { "6", 1 },
            { "7", 3 },
};
// MoveAbsJ: Perform joint movement to the fourth target point
```

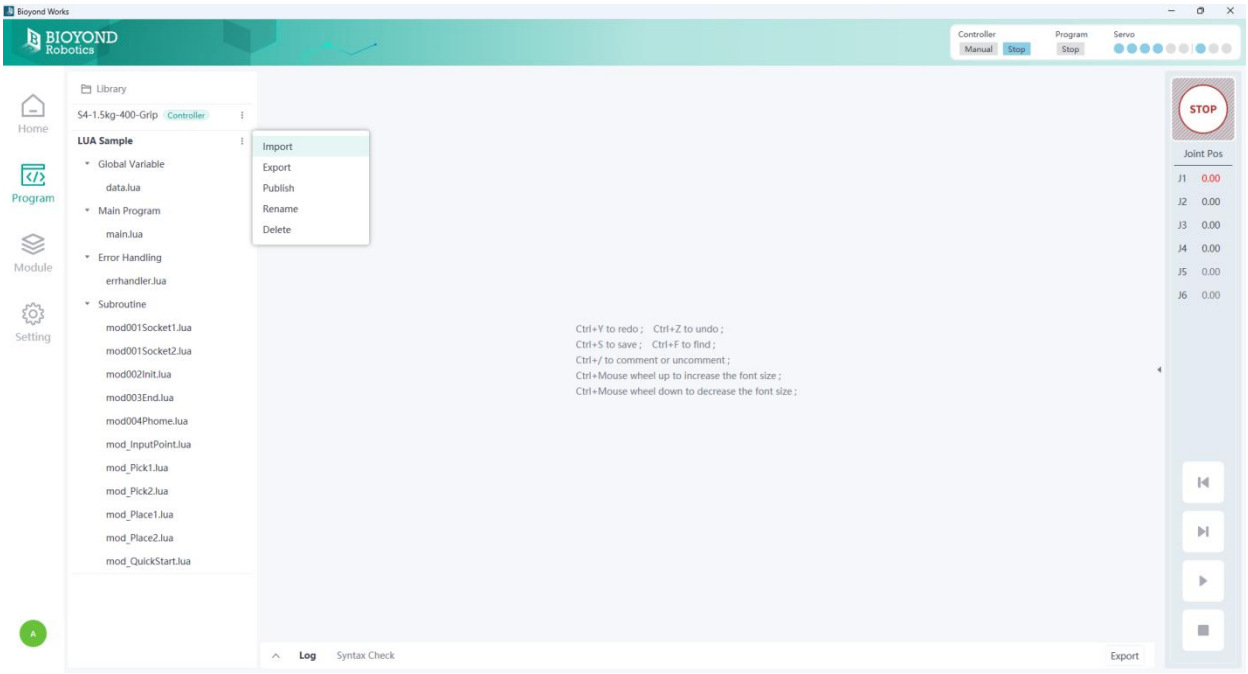
```
                robotDemo.SetSysParaF(setSysParaF4, 5 * 60 * 1000);
                robotDemo.SetSysVarI(setSysVal4, 5 * 60 * 1000);
            }
            Thread.Sleep(20000);

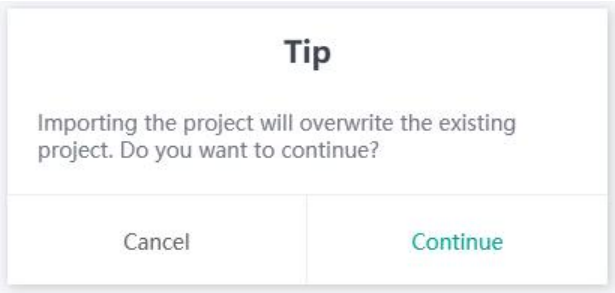
            //Close the connection with the robot
            robotDemo.Disconnect();
            Console.ReadLine();
        }
    }
}
```

5. Switching Between different APPLICATION SCENARIOS

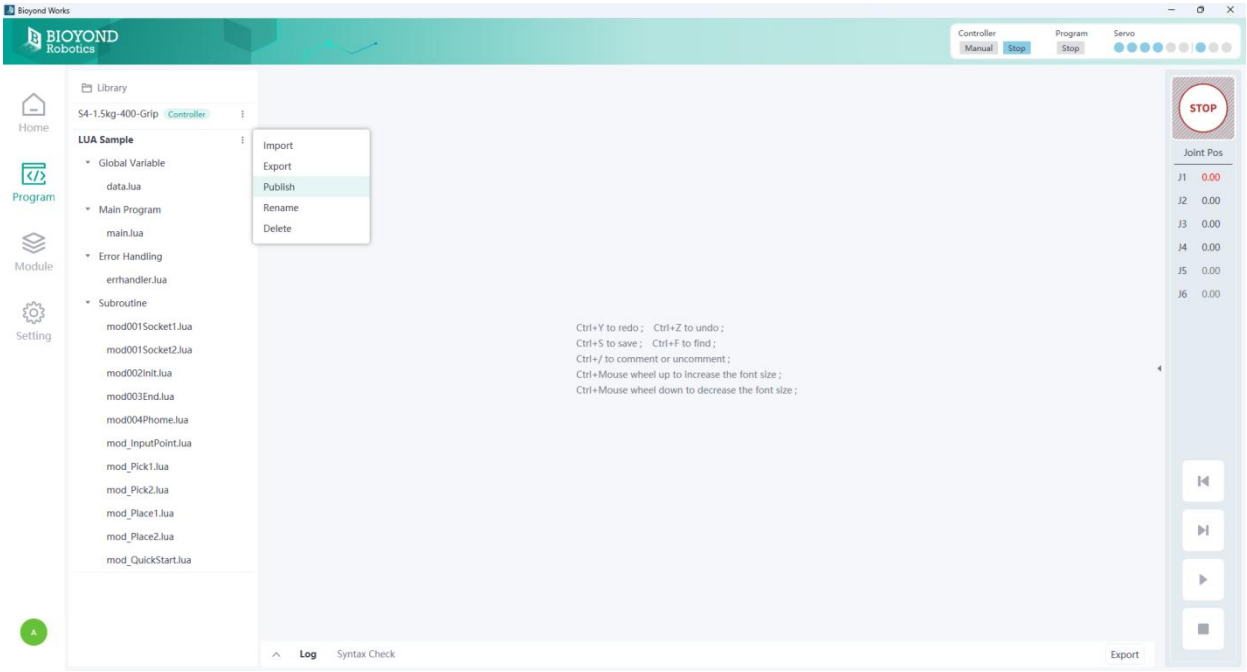
The Lua program should be published to the controller in advance through Bioyond Works. The specific steps are as follows:

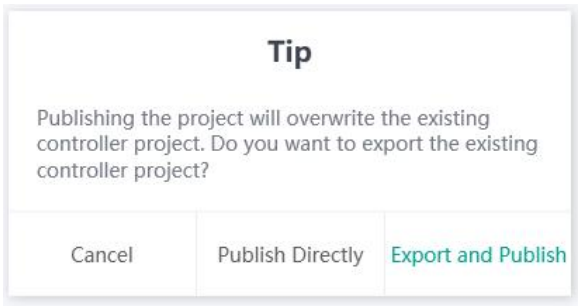
- Import LUA Sample.bin into the Offline Project. When importing the program into the offline project, it will overwrite the current offline project. You can back up the offline project by exporting it before importing the new program.





- Publish the Offline Project to the Controller. Publishing the offline project will overwrite the existing online project in the controller. When prompted, you can choose to back up the controller's existing program in advance.

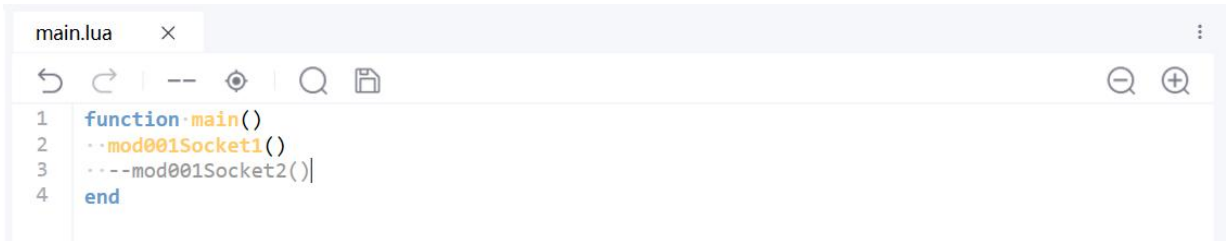


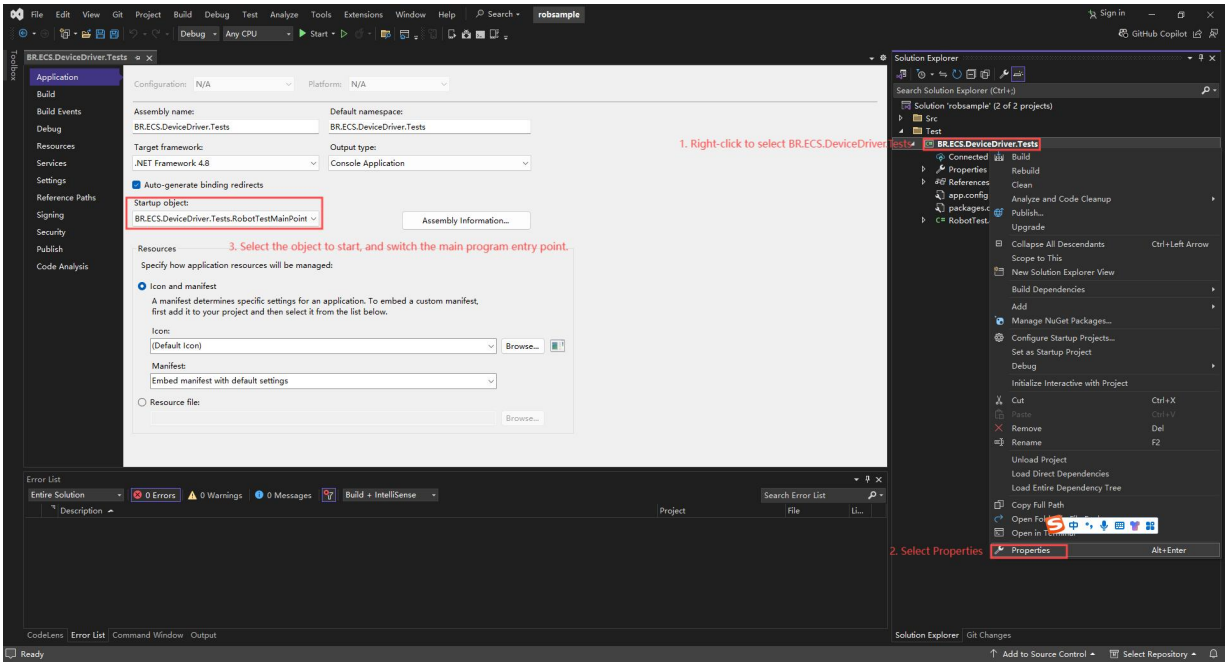


The robot program's main.lua and the automation control software's Main() need to run in coordination.

In Example 1, the main.lua calls the mod001Socket1(), and the automation control software starts with the object BR.ECS.DeviceDriver.Tests.RobotTestMainPointInBRTeach.

In Example 2, the main.lua calls the mod001Socket2(), and the automation control software starts with the object BR.ECS.DeviceDriver.Tests.RobotTestMainPointInUpperComputer.




1. Right-click to select BR.ECS.DeviceDriver.Tests
2. Select Properties
3. Select the object to start, and switch the main program entry point.
Startup object:
BR.ECS.DeviceDriver.Tests.RobotTestMainPoint
Properties
Alt+Enter